

Metodología de Sistemas I

Tecnicatura Universitaria en Programación

Ing. Landoni, Hernán

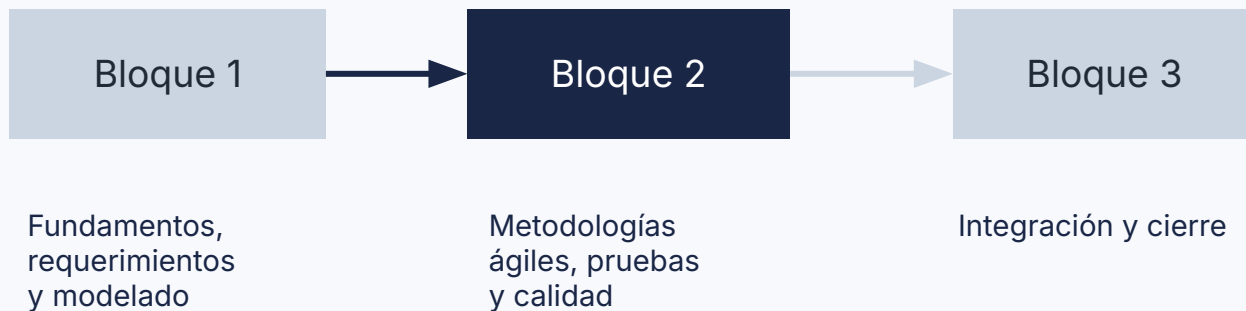
Bloque 2

Metodologías ágiles, pruebas y calidad

Clase 11

Del enfoque tradicional al Manifiesto Ágil.

¿Dónde estamos?



Desafíos de las metodologías tradicionales

Requerimientos

Los requerimientos cambian durante el proyecto

Documentación

La documentación queda desactualizada rápidamente

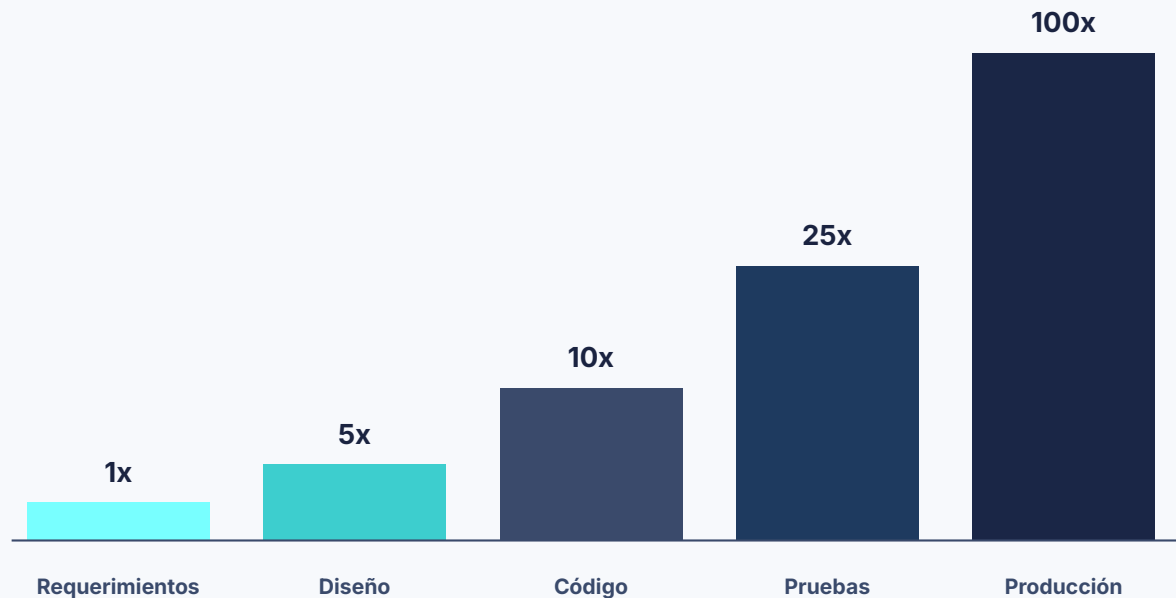
Costo del cambio

El costo del cambio crece exponencialmente en Waterfall

Feedback tardío

El cliente no ve software funcionando hasta el final

El costo del cambio



Fuente: Barry Boehm, Software Engineering Economics, 1981

La reunión

17 desarrolladores, consultores y metodólogos se reúnen en una estación de ski en Snowbird, Utah, Estados Unidos.

Representaban metodologías distintas: XP, Scrum, DSDM, Crystal, FDD, Pragmatic Programming.

La pregunta que los reunió:

"¿Hay una mejor manera de desarrollar software? ¿Podemos ponernos de acuerdo sobre qué principios comparten todos nuestros enfoques?"

El resultado:

El Manifiesto Ágil para el Desarrollo de Software.

Los 4 valores del Manifiesto Ágil



Individuos e interacciones

por sobre



Procesos y herramientas

Colaboración con el cliente

por sobre

Negociación contractual

Respuesta al cambio

por sobre

Seguir un plan

Software funcionando

por sobre

Documentación exhaustiva

"Estamos descubriendo mejores maneras de desarrollar software, tanto por nuestra propia experiencia como ayudando a terceros."

Valor 1: Individuos e interacciones por sobre procesos y herramientas

¿Qué significa?

- Un proceso rígido no reemplaza la comunicación directa entre personas.
- Una herramienta no compensa un equipo disfuncional.
- Los mejores resultados vienen de equipos que se comunican bien, no de los que siguen el proceso más detallado.

Ejemplo:

Dos desarrolladores que se hablan directamente resuelven un problema en 10 minutos.

El mismo problema documentado en un ticket y enviado por Jira puede tardar 2 días.

¿Qué no descarta?

- **Los procesos y las herramientas siguen siendo valiosos.**
- **No se eliminan:** se priorizan menos que las personas.

Valor 2: Software funcionando por sobre documentación exhaustiva

¿Qué significa?

- Un documento no es evidencia de progreso. El software que corre es evidencia de progreso.
- La documentación tiene valor cuando ayuda al equipo, no cuando existe por sí misma.
- Entregar software funcional frecuentemente genera feedback real que ningún documento puede reemplazar.

Ejemplo:

300 páginas de documentación y el cliente todavía no vio nada funcionar.

Con enfoque ágil se pueden entregar avances parciales para que se vea el sistema en funcionamiento y se obtenga feedback temprano.

¿Qué no descarta?

- **La documentación necesaria sigue siendo valiosa:** arquitectura, APIs, decisiones de diseño.
- **Lo que elimina es la documentación por protocolo, sin uso real.**

Valor 3: Colaboración con el cliente por sobre negociación contractual



Modelo tradicional

- Contrato define alcance fijo
- Cliente firma y desaparece
- Cambios requieren orden formal
- Conflicto cuando el resultado no es lo esperado



Modelo colaborativo

- El cliente es parte del equipo
- Participa en reviews periódicas
- Sus prioridades guían el backlog
- El resultado evoluciona con su feedback

¿Qué no descarta?

- Los contratos siguen siendo necesarios, especialmente en proyectos externos.
- Lo que cambia es la actitud: **el contrato no reemplaza la relación de trabajo.**

Valor 4: Respuesta al cambio por sobre seguir un plan

Dimensión	Tradicional	Ágil
Fuente del cambio	Error en la planificación	Aprendizaje y nueva información
Reacción típica	Freeze, change request, CCB	Ajustar el backlog, próximo sprint
Costo percibido	Alto, disruptivo	Esperado, manejable
Resultado	Software correcto según plan	Software útil según contexto actual

¿Qué no descarta?

- Los planes siguen siendo necesarios. **Planificar es valioso.**
- **Lo que cambia es la actitud frente al cambio:** no es una falla del proceso, es información nueva.

Los 12 principios: entrega y cliente

Principios 1 al 4

1

Satisfacción del cliente

La máxima prioridad es satisfacer al cliente mediante entregas tempranas y continuas de software con valor.

2

Bienvenida al cambio

Los cambios en los requerimientos son bienvenidos, incluso tardíos. Los procesos ágiles aprovechan el cambio como ventaja competitiva.

3

Entrega frecuente

Entregar software funcionando frecuentemente, en ciclos de pocas semanas, preferiblemente cortos.

4

Trabajo conjunto

Las personas de negocio y los desarrolladores deben trabajar juntos a diario durante todo el proyecto.

Los 12 principios: equipo y proceso

Principios 5 al 8

5

Motivación del equipo

Construir proyectos en torno a individuos motivados. Darles el entorno y el apoyo que necesitan, y confiar en que harán el trabajo.

6

Conversación cara a cara

El método más eficiente y efectivo de transmitir información es la conversación cara a cara.

7

Software funcionando

El software funcionando es la medida principal de progreso. (Refuerza el Valor 2.)

8

Ritmo sostenible

Los procesos ágiles promueven el desarrollo sostenible. El equipo debe poder mantener un ritmo constante indefinidamente.

Los 12 principios: técnica y reflexión

Principios 9 al 12

9

Excelencia técnica

La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.

10

Simplicidad

La simplicidad, el arte de maximizar el trabajo que no se hace, es esencial.

11

Equipos autoorganizados

Las mejores arquitecturas, requerimientos y diseños emergen de equipos autoorganizados.

12

Reflexión y ajuste

A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y ajusta su comportamiento en consecuencia.

¿Cuándo usar cada enfoque?

Dimensión	Tradicional (Waterfall / V / RUP)	Ágil (Scrum / Kanban / XP)
Requerimientos	Estables, bien definidos desde el inicio	Cambiantes, emergen durante el proyecto
Cliente	Disponible al inicio y al final	Involucrado continuamente
Equipo	Especialistas en roles fijos	Multifuncional, autoorganizado
Entregables	Un entregable final de gran escala	Entregas frecuentes, incrementales
Contrato	Precio fijo, alcance fijo	Tiempo y materiales, alcance flexible
Contexto típico	Gobierno, defensa, sistemas críticos, grandes contratos	Producto digital, startups, equipos internos

Deuda técnica

Concepto acuñado por Ward Cunningham (uno de los firmantes del Manifiesto) en 1992.

"Cada vez que tomás un atajo en el código para entregar más rápido, estás tomando un préstamo contra el futuro. Con intereses."

Deuda deliberada

Atajo consciente para cumplir un deadline.
Aceptable si se registra y se paga.

Deuda accidental

Resultado de malas prácticas o falta de conocimiento. No fue una decisión consciente.

Deuda por entropía

El sistema envejece y el código que era correcto deja de serlo con el tiempo.

Críticas a las metodologías ágiles

"Ágil se convirtió en una industria de certificaciones, consultores y frameworks pesados. Exactamente lo que el Manifiesto intentó reemplazar."

Certificaciones como fin en sí mismo

CSM, SAFe, PMP ágil: el mercado capturó el movimiento y lo convirtió en burocracia certificable.

SAFe y la escala corporativa

Scaled Agile Framework: 130+ páginas de proceso. Irónicamente más pesado que Waterfall para muchos equipos.

Cargo cult agile

Equipos que hacen standups, sprints y retrospectivas sin entender por qué. La ceremonia sin el espíritu.

El Manifiesto hoy: 25 años después

Firmado en febrero de 2001. Sigue online en agilemanifesto.org.

Lo que se cumplió

Ágil dominó la industria del software comercial. El 71% de los equipos usa algún framework ágil (State of Agile Report 2023).

Lo que cambió

El ecosistema creció: SAFe, LeSS, Nexus, Shape Up. Cada organización adaptó los principios a su contexto.

Lo que se distorsionó

El término 'ágil' se diluyó. Equipos que llaman ágil a cualquier cosa sin planificación o disciplina técnica.

Lo que sigue vigente

Los 4 valores y 12 principios no envejecieron. El documento original, de 68 palabras, sigue siendo la referencia.

Lo ágil en la práctica: casos reales

Spotify



Squads, Tribes, Chapters, Guilds

Equipos de 6 a 12 personas (squads) con autonomía completa sobre un área. Escalado a cientos de equipos sin perder velocidad. El modelo se publicó en 2012 y fue adoptado por decenas de empresas.

~2.000 squads activos

Amazon



Two-Pizza Teams

Jeff Bezos: si un equipo necesita más de dos pizzas para comer, es demasiado grande. Equipos de 5 a 8 personas, autónomos, con ownership completo de su servicio. Esto impulsó directamente la arquitectura de microservicios de AWS.

Regla del equipo: max 8 personas

ING Bank



Transformación ágil corporativa

En 2015, ING reorganizó 3.500 empleados en squads y tribus, inspirándose en el modelo Spotify. Redujo time-to-market de meses a semanas. Es uno de los casos más citados de adopción ágil en banca.

3.500 empleados reorganizados en 2015

Lo ágil en la práctica: casos reales

En un equipo tradicional

Recibís especificaciones detalladas y las implementás.

Tu trabajo termina cuando el código compila y pasa las pruebas unitarias.

Los cambios de requerimientos son una interrupción del plan.

La comunicación con el cliente pasa por el analista o el PM.

Sos responsable de tu módulo, no del sistema completo.

En un equipo ágil

Participás en la definición del backlog y en la estimación de tareas.

Sos responsable de entregar valor en cada sprint, no solo código.

El cambio es parte del trabajo, no una excepción.

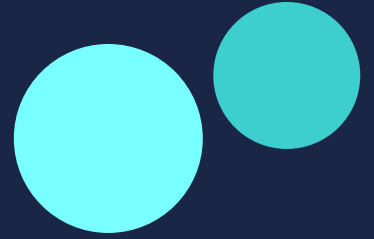
Interactuás directamente con el cliente o Product Owner.

El equipo tiene ownership colectivo: el código de todos es tuyo.

Actividad de cierre

Escenario: el sistema de turnos médicos está a mitad de desarrollo con Waterfall. El cliente llama y comunica que quiere agregar un módulo de teleconsultas y cambiar la lógica de cancelación de turnos.

- 1** **Impacto en Waterfall**
¿Qué pasos requiere gestionar estos cambios en un proceso Waterfall? ¿Qué documentos se ven afectados?
- 2** **Análisis con el Manifiesto**
¿Qué valores y principios del Manifiesto Ágil son directamente relevantes para este escenario? Identificá al menos dos.
- 3** **Comparación**
¿Cómo manejaría este cambio un equipo con enfoque ágil? ¿Qué diferencias concretas identificás?



Clase 12

SCRUM

¿Qué es Scrum?

"Scrum es un framework liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos."

Scrum Guide 2020 - Ken Schwaber y Jeff Sutherland

- **Origen:** Sutherland formalizan Scrum en 1995.
- **No es una metodología** ni un proceso: **es un marco de trabajo** (*framework*). Define estructura mínima, no procedimientos detallados.
- **Adopción:** 71% de los equipos ágiles usa Scrum o una variante (State of Agile Report 2023).
- Scrum se apoya en **tres pilares** y **cinco valores** que definen cómo trabaja el equipo.

Los 3 pilares de Scrum

Transparencia

El proceso y el trabajo deben ser visibles para todos los involucrados. No hay información reservada. El backlog, el progreso y los impedimentos son públicos dentro del equipo.

Inspección

Los artefactos y el progreso se inspeccionan con frecuencia para detectar variaciones indeseadas. Las ceremonias de Scrum son los puntos de inspección formales.

Adaptación

Si la inspección revela desviaciones, el proceso o el producto se ajustan. Sin adaptación, la inspección no tiene valor. La Retrospectiva es el mecanismo formal de adaptación del proceso.

Los 5 valores de Scrum

Compromiso

El equipo se compromete a alcanzar sus objetivos y a apoyarse mutuamente.

Foco

El foco está en el trabajo del Sprint y en los objetivos del equipo.

Apertura

El equipo y sus stakeholders son abiertos sobre el trabajo y los desafíos.

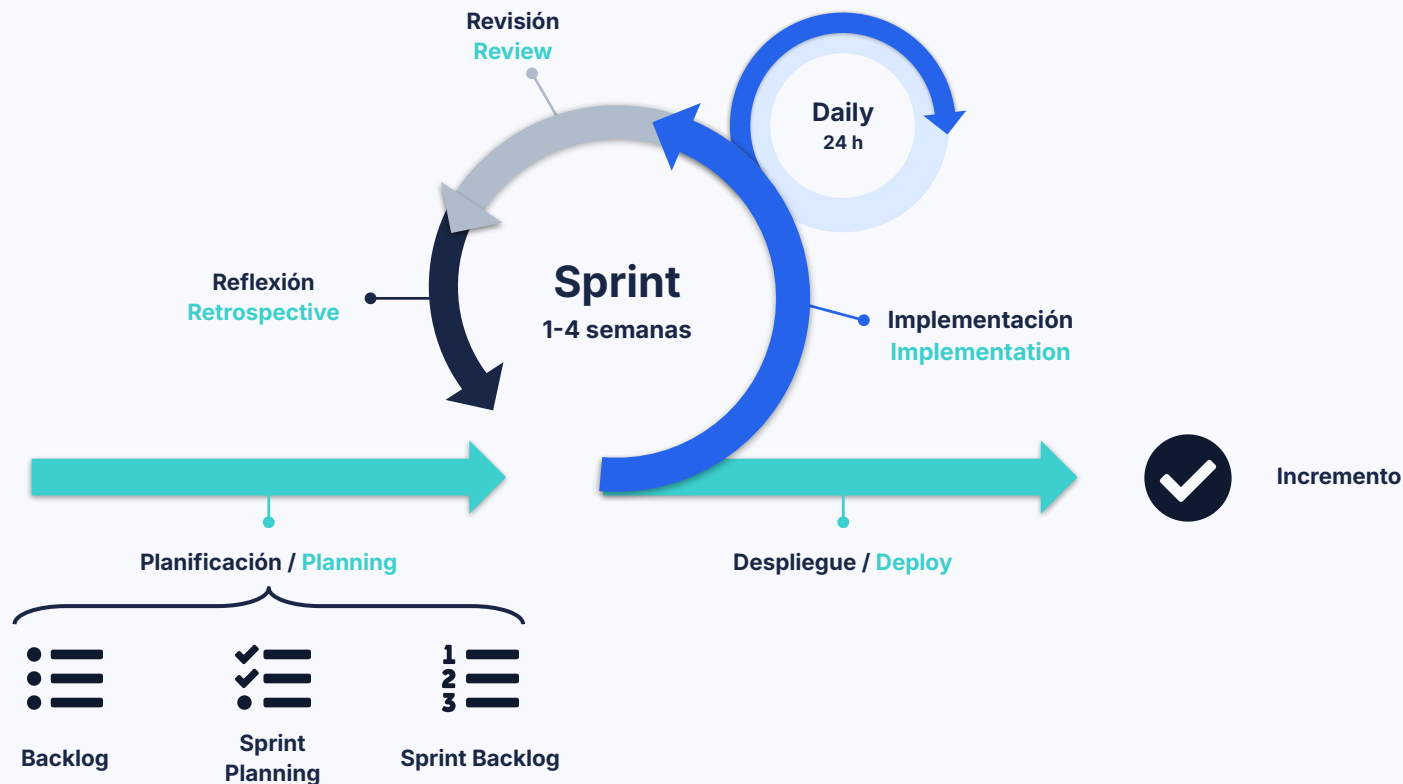
Respeto

Los miembros del equipo se respetan como personas capaces e independientes.

Coraje

El equipo tiene coraje para hacer lo correcto y trabajar en problemas difíciles.

El ciclo Scrum: visión general



Rol: Product Owner

Responsable de maximizar el valor del producto resultante del trabajo del equipo.

Responsabilidades:

- Gestiona y prioriza el Product Backlog.
- Define el Sprint Goal junto al equipo.
- Representa al cliente y a los stakeholders.
- Acepta o rechaza el trabajo al final del Sprint.

Perfil real en la industria:

- Puede ser el cliente, un representante interno, o un Product Manager.
- En startups suele ser el CEO o el fundador en etapas tempranas.

Errores frecuentes:

- **PO ausente:** el equipo no tiene a quién consultar y toma decisiones de negocio sin autoridad.
- **PO que microgestiona:** convierte el backlog en una lista de tareas técnicas en lugar de historias de valor.
- **Múltiples POs:** genera conflictos de prioridad. Scrum es explícito: hay uno solo.

Rol: Scrum Master

Responsable de la efectividad del equipo Scrum. Facilita, protege y remueve impedimentos.

Responsabilidades:

- Facilita las ceremonias de Scrum.
- Remueve impedimentos que bloquean al equipo.
- Protege al equipo de interrupciones externas.
- Entrena al equipo y a la organización en Scrum.
- Es un líder servidor, no un jefe.

Lo que el Scrum Master NO es:

- No es un Project Manager: no controla el cronograma ni reporta avance a la gerencia.
- No es un secretario del equipo: no toma notas, no agenda reuniones.
- No es un desarrollador part-time: el rol requiere dedicación.

Impedimentos reales:

- Acceso bloqueado a un ambiente de pruebas.
- Dependencia de otro equipo que no responde.
- Requerimiento ambiguo que el PO no aclara.

Errores frecuentes:

- SM que reporta a la gerencia en lugar de proteger al equipo.
- SM que resuelve los impedimentos por el equipo en lugar de enseñarle a resolverlos.

Rol: Equipo de Desarrollo

Profesionales que transforman los ítems del Sprint Backlog en un Increment usable al final de cada Sprint.

Características:

- **Multifuncional:** tiene todas las habilidades para entregar el Increment.
- **Autoorganizado:** decide cómo hacer el trabajo.
- **Sin roles internos fijos:** no hay líder técnico formal.
- **Tamaño:** entre 3 y 9 personas.
- Ownership colectivo del código.

Qué cambia para el programador:

- Participa en la estimación de tareas (Planning Poker).
- Puede hablar directamente con el Product Owner.
- Responsable de la calidad técnica del Increment.
- Participa en la Retrospectiva y tiene voz sobre el proceso.

Tensión real:

- Autoorganización requiere madurez del equipo.
- Equipos nuevos necesitan más guía inicial.

Artefacto: Product Backlog

Lista ordenada de todo lo que se necesita en el producto. La única fuente de trabajo para el equipo Scrum.

Características:

- Es dinámico: nunca está completo. Evoluciona con el producto.
- El Product Owner es el único responsable de su contenido y orden.
- Los ítems más prioritarios están más detallados.
- Contiene funcionalidades, mejoras, correcciones y deuda técnica.

Refinamiento del backlog:

- Actividad continua de descomposición y estimación de ítems futuros.
- Consume hasta el 10% del tiempo del equipo.

Ejemplo: sistema de turnos médicos

Alta	El paciente puede registrarse con email y contraseña
Alta	El paciente puede ver turnos disponibles por especialidad
Alta	El paciente puede reservar un turno
Media	El médico puede ver su agenda del día
Media	El sistema envía recordatorio 24 hs antes
Baja	El paciente puede solicitar una teleconsulta

Artefacto: Sprint Backlog

Conjunto de ítems del Product Backlog seleccionados para el Sprint, más el plan para entregarlos.

Características:

- Propiedad exclusiva del equipo de desarrollo.
- Solo el equipo puede modificarlo durante el Sprint.
- Debe ser visible para todos: tablero físico o digital.
- Refleja el trabajo restante en tiempo real.

Diferencia con el Product Backlog:

- Product Backlog: todo lo que podría hacerse.
Owned by PO.
- Sprint Backlog: lo que se hará en este Sprint.
Owned by el equipo.

Tablero del Sprint

Por hacer	En progreso	Hecho
Registro de paciente	Login con email	Modelo de BD
Ver turnos por especialidad		Setup del entorno

Artefacto: Incremento y Definition of Done

El Incremento es la suma de todos los ítems completados en el Sprint más todos los Sprints anteriores. Debe ser usable.

Incremento:

- Debe ser funcional al final de cada Sprint.
- Cada Sprint produce al menos un Increment.
- Es aditivo: se construye sobre los Incrementos anteriores.

Definition of Done (DoD):

- Criterio formal que define cuándo un ítem está realmente terminado.
- Si un ítem no cumple la DoD, no forma parte del Increment.
- Ejemplo de DoD mínima:
 - Código mergeado en main.
 - Tests unitarios escritos y pasando.
 - Code review aprobado.
 - Desplegado en staging.

Por qué importa la DoD:

- Sin DoD, 'terminado' significa algo diferente para cada uno.
- Una DoD débil acumula deuda técnica sprint a sprint.
- Una DoD fuerte hace que los sprints sean predecibles.

Errores frecuentes:

- 'Terminado' = el código funciona en mi máquina.
- 'Terminado' = el ticket está cerrado en Jira.
- 'Terminado' = pasaron los tests pero no hay code review.

El Sprint como contenedor

El Sprint es el corazón de Scrum. Un evento de duración fija de máximo 4 semanas donde se crea valor.

Características del Sprint:

- **Duración fija:** 1, 2, 3 o 4 semanas. La más común: 2 semanas.
- **Contiene todas las ceremonias:** Planning, Daily, Review, Retrospective.
- **El Sprint Goal define el objetivo:** por qué se hace este Sprint.

Reglas del Sprint:

- No se cambia el Sprint Goal una vez iniciado.
- No se agrega trabajo sin acuerdo del equipo.
- Solo el PO puede cancelar un Sprint: situación excepcional.

Sprint Goal:

- No es una lista de tareas. Es el objetivo de negocio del Sprint.
- **Débil:** 'terminar las HU 12, 13 y 14'.
- **Fuerte:** 'el paciente puede registrarse y ver su agenda'.

Por qué importa la duración fija:

- **Crea cadencia:** el equipo aprende a predecir su velocidad.
- **Limita el daño:** si algo sale mal, el Sprint termina en días.
- **Genera ritmo organizacional:** stakeholders saben cuándo habrá demo.

Ceremonia: Sprint Planning

Evento que inicia el Sprint. El equipo define qué se hará y cómo se hará.

Parámetros:

- Duración máxima: 2 horas por semana de Sprint.
- Participantes: todo el equipo Scrum.

Outputs:

- Sprint Goal definido.
- Sprint Backlog con los ítems seleccionados.
- Plan inicial de trabajo (tareas técnicas).

Las 3 preguntas del Sprint Planning:

- ¿Por qué es valioso este Sprint? → Sprint Goal.
- ¿Qué se puede completar en este Sprint? → Selección del backlog.
- ¿Cómo se hará el trabajo seleccionado? → Plan técnico.

Errores frecuentes:

- Product Backlog sin refinar: el equipo llega sin entender los ítems. El Planning se convierte en análisis de requerimientos.
- Sobrecomprometer: el equipo acepta más trabajo del que puede completar. Genera sprints fallidos recurrentes.
- Sin Sprint Goal: el Sprint es solo una lista de tareas sin propósito unificador.

Buena práctica:

- Refinar el backlog continuamente durante el Sprint anterior. Así el Planning es fluido.

Ceremonia: Daily Scrum

Evento diario de 15 minutos para inspeccionar el progreso hacia el Sprint Goal y adaptar el Sprint Backlog.

Parámetros:

- Duración fija: 15 minutos.
- Mismo horario y lugar todos los días.
- Solo el equipo de desarrollo. El SM facilita si es necesario.

Las 3 preguntas clásicas:

- ¿Qué hice ayer que ayudó al Sprint Goal?
- ¿Qué haré hoy para ayudar al Sprint Goal?
- ¿Hay algún impedimento que bloquee mi avance?
- Nota: la Scrum Guide 2020 no obliga estas preguntas.

Errores frecuentes:

- Reporte de estado al SM: 'ayer hice X, hoy haré Y'. La Daily no es para el SM, es para el equipo.
- Daily de 45 minutos: las discusiones técnicas deben agendarse aparte.
- Impedimentos sin resolución: si el SM no resuelve los impedimentos, la Daily pierde credibilidad.
- Daily sin tablero visible: el equipo habla en abstracto.

Valor real:

- La Daily detecta bloqueos en 24 horas, no en una semana.

Ceremonia: Sprint Review

Evento al final del Sprint donde el equipo presenta el Increment y recibe feedback de los stakeholders.

Parámetros:

- Duración máxima: 1 hora por semana de Sprint.
- Participantes: equipo Scrum completo más stakeholders.
- Se muestra software funcionando, no slides.

¿Qué pasa en la Review?:

- El equipo demuestra el Incremento sobre el entorno real.
- Los stakeholders dan feedback.
- El PO actualiza el Product Backlog según lo aprendido.
- Se discuten las próximas prioridades.

Diferencia con la Retrospectiva:

- Review: inspecciona el producto. ¿Estamos construyendo lo correcto?
- Retrospectiva: inspecciona el proceso. ¿Estamos trabajando bien?

Errores frecuentes:

- Review sin stakeholders reales: no hay feedback nuevo.
- Demo de features incompletas: viola la DoD y erosiona la confianza.
- Review convertida en presentación de slides: el feedback sobre slides no es feedback sobre el producto.

Ceremonia: Sprint Retrospective

Evento donde el equipo inspecciona cómo trabajó y define al menos una mejora concreta para el próximo Sprint.

Parámetros:

- Duración máxima: 45 minutos por semana de Sprint.
- Solo el equipo Scrum. Los stakeholders no participan.
- El SM facilita y garantiza un espacio seguro.

Estructura clásica (Start/Stop/Continue):

- ¿Qué deberíamos empezar a hacer?
- ¿Qué deberíamos dejar de hacer?
- ¿Qué deberíamos seguir haciendo?
- Output obligatorio: al menos una mejora accionable.

Errores frecuentes:

- Retrospectiva sin cambios: el equipo identifica problemas pero no implementa mejoras. La retro pierde credibilidad.
- Retro como sesión de quejas: sin estructura, se convierte en una lista de frustraciones sin acciones.
- Ausencia del SM: sin facilitación, el equipo no llega a conclusiones concretas.

Por qué es el evento más valioso:

- Es el único espacio formal donde el equipo puede mejorar su propio proceso.

Estimación: story points y Planning Poker

Los story points estiman el esfuerzo relativo de un ítem, no el tiempo absoluto. Capturan complejidad, incertidumbre y volumen.

¿Por qué no horas?

- **Las horas varían por persona:** 4 horas para un senior son 2 días para un junior.
- **El tiempo crea falsa precisión:** decir "6 horas" parece más exacto que lo que realmente es.
- **Los story points son relativos:** si una tarea es "2", una más compleja es "5" o "8".

Escala de Fibonacci:

- **1, 2, 3, 5, 8, 13, 21.** Los gaps aumentan con la incertidumbre.
- **Un ítem de 13 o más puntos suele necesitar descomposición.**

Planning Poker:

- Cada miembro del equipo estima independientemente y revela al mismo tiempo.
- Si hay divergencia, quienes dieron el valor más alto y más bajo explican su razonamiento.
- El proceso de discusión es más valioso que el número final.
- Herramientas: planningpoker.com, cartas físicas, extensiones de Jira.

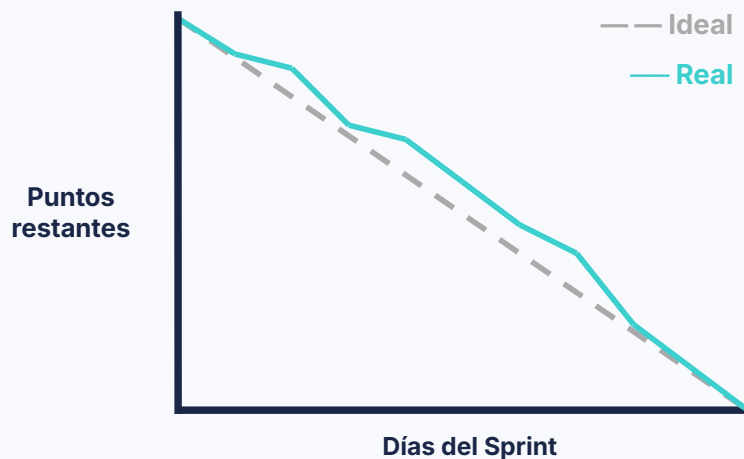
Velocidad del equipo:

- Promedio de story points completados por Sprint.
- Después de 3 a 4 sprints, la velocidad es un indicador de planificación confiable.

Métricas: burndown chart y velocidad

El burndown chart muestra el trabajo restante en el Sprint. La velocidad mide la capacidad sostenible del equipo.

Burndown chart para un Sprint de 10 días



Cómo leer el burndown:

- **Línea ideal:** pendiente constante de inicio a fin del Sprint.
- **Línea real por encima de la ideal:** el equipo va más lento de lo esperado.
- Línea real plana: trabajo bloqueado o no actualizado en el tablero.

Velocidad:

- Promedio de puntos completados en los últimos 3 a 5 Sprints.
- No es una métrica de productividad individual. Es una referencia de planificación del equipo.
- **Manipulación frecuente:** inflar las estimaciones para mostrar alta velocidad. El resultado es pérdida de significado.

Scrum aplicado: Product Backlog del sistema de turnos

Product Backlog inicial (priorizado por el Product Owner)

#	Épica	Historia de usuario	Pts
1	Acceso	Como paciente, quiero registrarme con email y contraseña para acceder al sistema.	3
2	Acceso	Como paciente, quiero iniciar sesión para ver mi información.	2
3	Turnos	Como paciente, quiero ver turnos disponibles por especialidad para elegir el que más me convenga.	5
4	Turnos	Como paciente, quiero reservar un turno para confirmar mi atención.	3
5	Agenda	Como médico, quiero ver mi agenda del día para organizar mis consultas.	5
6	Notif.	Como paciente, quiero recibir un recordatorio 24 hs antes de mi turno.	8

Scrum aplicado: Sprint 1 y Sprint Goal

Primer Sprint del sistema de turnos: del Product Backlog al Incremento funcionando.

Sprint Goal:

"Al final del Sprint, un paciente puede registrarse e iniciar sesión en el sistema."

Sprint Backlog seleccionado:

- HU-1: Registro con email y contraseña (3 pts).
- HU-2: Login de paciente (2 pts).
- Total: 5 story points. Equipo nuevo, Sprint conservador.

Tareas técnicas derivadas:

- Diseño del modelo de datos: tabla Paciente.
- Endpoint POST /registro y POST /login.
- Pantalla de registro y login (frontend).
- Tests unitarios de validación de campos.
- Deploy en staging.

Roles en este Sprint:

- Product Owner: clínica o representante. Disponible para consultas sobre el flujo de registro.
- Scrum Master: garantiza que el equipo no sea interrumpido. Gestiona acceso al ambiente.
- Equipo: 3 desarrolladores. Autoorganizados para dividir las tareas.

Sprint Review esperada:

- Demo en vivo: el representante de la clínica se registra y loguearse en el sistema real.
- Feedback probable: ¿se puede registrar con Google? ¿Se requiere verificación de email?
- Esos ítems van al backlog para el Sprint 2.

Errores frecuentes en equipos Scrum

La mayoría de los problemas con Scrum no son del framework. Son de implementación.

01

Sin Product Owner real

El PO es alguien que no tiene autoridad para tomar decisiones o que no está disponible. El equipo pierde tiempo esperando respuestas o toma decisiones de negocio sin estar habilitado para eso.

02

Dailies como reporte de estado

El equipo le reporta al SM o al manager en lugar de coordinarse entre sí. Síntoma: el equipo habla en primera persona singular ('yo hice', 'yo haré') en lugar de en función del Sprint Goal.

03

Sprints sin Definition of Done

'Terminado' significa algo diferente para cada uno. La deuda técnica se acumula sprint a sprint hasta que el sistema se vuelve inmanejable.

04

Retrospectivas sin cambios

El equipo identifica los mismos problemas sprint a sprint pero no implementa mejoras. La retro pierde credibilidad y el equipo deja de participar genuinamente.

05

Scrum sin disciplina técnica

Sprints cortos con código de baja calidad generan deuda técnica rápidamente. Ágil sin excelencia técnica no es sostenible. (Principio 9 del Manifiesto.)

Actividad de cierre

Escenario: el equipo de desarrollo del sistema de turnos necesita preparar su segundo sprint

Consigna (grupos de 3 a 4 personas):

1

Definir el Product Backlog

Escribir 6 historias de usuario priorizadas para el sistema de turnos. Asignar story points usando la escala de Fibonacci. Identificar al menos una épica.

2

Planificar el Sprint 1

Seleccionar las historias que entrarían en el primer Sprint de 2 semanas. Definir el Sprint Goal. Estimar la capacidad del equipo (3 personas, 2 semanas).

3

Definir la Definition of Done

Escribir la DoD del equipo: ¿qué condiciones debe cumplir un ítem para considerarse terminado?

Clase 13

Lean y Kanban

Origen de Lean: Toyota Production System

Taiichi Ohno, Japón post-guerra. La pregunta: ¿cómo producir más con menos?

El contexto

Japón, 1940s: recursos escasos. No podía competir con el volumen de producción de Ford.

La respuesta

Taiichi Ohno diseñó el Toyota Production System (TPS): eliminar todo lo que no agrega valor.

Just-in-time

Producir solo lo que se necesita, cuando se necesita, en la cantidad necesaria.

Jidoka

Automatización con toque humano: detener la línea cuando hay un defecto. Calidad integrada al proceso.

El resultado

Toyota pasó de casi quiebra en los 50s a ser el mayor fabricante de autos del mundo. El TPS fue el diferenciador clave.

La traducción al software

Mary y Tom Poppendieck, 2003. Los principios de manufactura sin desperdicio aplicados al desarrollo de software. El 'producto' es el software funcionando.

Los 7 principios de Lean Software Development

Poppendieck, 2003. La traducción de los principios del TPS al desarrollo de software.

01	Eliminar el desperdicio	Todo lo que no agrega valor directo al cliente.
02	Amplificar el aprendizaje	Iteraciones cortas para maximizar el conocimiento.
03	Decidir lo más tarde posible	Posponer decisiones irreversibles hasta tener más información.
04	Entregar lo más rápido posible	Ciclos cortos reducen el costo del cambio.
05	Empoderar al equipo	Quienes hacen el trabajo saben mejor cómo mejorarlo.
06	Construir con integridad	La integridad conceptual importa tanto como la técnica.
07	Ver el todo	Optimizar una parte sin ver el flujo completo puede empeorar el resultado.

Los 7 desperdicios en software (1/2)

El trabajo existe pero no fluye hacia el cliente.

Trabajo parcialmente hecho

- 01** Iniciado pero no entregado. No genera valor hasta completarse.

Ejemplo

Una feature 3 sprints en el tablero sin *mergearse*. El código existe pero no aporta nada.

Espera

- 02** El trabajo está bloqueado esperando algo externo.

Ejemplo

Un PR esperando code review 3 días. Una tarea bloqueada por una dependencia que no responde.

Cambios de contexto

- 03** Costo cognitivo de alternar entre tareas.

Ejemplo

Un desarrollador en 3 proyectos simultáneos pierde 15-20 min por cambio de contexto (Weinberg, 1992).

Movimiento

- 04** Transferencia de información con pérdida en cada traspaso.

Ejemplo

Cliente → PM → Analista → Desarrollador: el desarrollador implementa lo que llegó, no lo que el cliente quería.

Los 7 desperdicios en software (2/2)

Retrabajo, burocracia y valor que nunca llega al usuario.

Defectos

- 05** Errores que hay que encontrar, reportar y corregir. Doble trabajo.

Ejemplo

Un bug en producción que requiere hotfix, regresión y redeploy. Cuanto más tarde se detecta, más costoso.

Procesos extras

- 06** Pasos que no agregan valor. Burocracia, reuniones innecesarias.

Ejemplo

Documentación que nadie lee. Reuniones de status que duplican lo que está en Jira.

Funcionalidades no usadas

- 07** Features construidas que el usuario no usa. El peor desperdicio: invisible.

Ejemplo

El 64% de las features de un producto típico se usan raramente o nunca (Pendo, 2023).

Teoría de las restricciones (Theory of Constraints)

Eliyahu Goldratt, 1984. La velocidad del sistema la determina su cuello de botella.

"La cadena es tan fuerte como su eslabón más débil. Optimizar cualquier parte que no sea el cuello de botella no mejora el resultado total."

El principio

Todo sistema tiene al menos un cuello de botella. La única forma de mejorar el throughput es identificarlo y resolverlo.

Los 5 pasos

Identificar → Explotar → Subordinar → Elevar → Repetir. Cuando se resuelve un cuello, aparece el siguiente.

En software

Si el cuello de botella es el code review, más desarrolladores no aceleran el delivery. Primero encontrar dónde se acumula el trabajo.

Conexión con Kanban

Los WIP limits hacen visible el cuello de botella: la columna que siempre está llena. Sin WIP limits, el cuello se oculta detrás del WIP acumulado.

Mapa del flujo de valor (Value stream mapping)

Herramienta para visualizar el flujo completo e identificar dónde se pierde tiempo.

¿Qué mapea?

Todos los pasos desde que el cliente solicita hasta que recibe.
Para cada paso: tiempo de proceso (valor) y tiempo de espera (desperdicio).

Cómo se usa

Mapear el estado actual → Identificar desperdicios → Diseñar el estado futuro → Implementar mejoras en el cuello de botella.

Ejemplo: feature request → producción

Solicitud 1d proc	Refinamiento 3d proc	Desarrollo 5d proc	Code Review 1d proc	QA 2d proc	Deploy 0.5d
0d espera	2d espera	1d espera	3d espera	1d espera	2d espera

Code Review: 3d espera vs. 1d proceso → cuello de botella visible.

Kaizen: mejora continua

改善 · “cambio para bien”. No grandes revoluciones: pequeños cambios sostenidos.

El principio

La mejora es un proceso continuo, no un evento. Cada persona en cada nivel busca activamente oportunidades de mejora.

PDCA (Deming)

Plan → Do → Check → Act. Implementar en pequeña escala, medir, estandarizar si funcionó.

El dato

Toyota implementa más de un millón de mejoras Kaizen al año. La mayoría son mínimas, pero el efecto acumulado es enorme.

Mejora tradicional

Proyectos grandes, esporádicos, top-down. La organización decide qué mejorar.

Kaizen

Cambios pequeños, continuos, bottom-up. El equipo que hace el trabajo identifica las mejoras.

En software

La Retrospectiva de Scrum es Kaizen formal. El refinamiento del backlog es Kaizen del producto.

De Lean a Kanban

Kanban operacionaliza los principios Lean con herramientas concretas.

Eliminar el desperdicio	→	WIP limits	Limitar el trabajo en progreso hace visible el desperdicio.
Ver el todo	→	Tablero Kanban	El tablero hace visible el flujo completo en un solo lugar.
Decidir lo más tarde posible	→	Flujo pull	El trabajo se jala cuando hay capacidad, no se empuja.
Entregar lo más rápido posible	→	Cadencia continua	Se entrega cuando está listo, no cuando termina el sprint.
Amplificar el aprendizaje	→	Retrospectivas / Kaizen	Revisión continua para identificar cuellos de botella.

Origen de Kanban en software

David Anderson, Microsoft, 2004. El primer experimento resolvió un backlog descontrolado en semanas.

El experimento

Microsoft XIT, 2004. Equipo de soporte con backlog desbordado. Anderson aplicó WIP limits y un tablero visual.

vs. Toyota

Toyota usa tarjetas físicas para inventario. Anderson tomó la señal visual y la adaptó: en software el 'inventario' es el WIP.

2007 →

Anderson publica el Kanban Method. Adopción en equipos de TI y soporte.

2010 →

Kanban se expande a equipos de desarrollo. Alternativa a Scrum para flujo continuo.

El resultado

En 6 meses el lead time se redujo de semanas a días. El experimento se publicó como 'Kanban for Software' en 2007.

Hoy →

Adoptado en +70 países. Equipos de desarrollo, operaciones, marketing. Herramientas: Jira, Linear, Trello, GitHub Projects.

El tablero Kanban

Representación visual del flujo. Cada columna es un estado. Cada tarjeta es una unidad de trabajo.

Componentes

Columnas (estados del flujo), tarjetas (unidades de trabajo), WIP limit (máximo por columna), swimlanes opcionales por tipo de trabajo.

Pendiente —	En análisis WIP: 1	Desarrollo WIP: 2	Review WIP: 2	Hecho —
Módulo reportes	Login OAuth	API turnos	Reg. paciente	Modelo BD
Exportar PDF		Vista médico		

WIP limit

El número máximo de tarjetas permitidas en una columna. Cuando se alcanza, el equipo debe terminar antes de empezar.

Tablero Kanban vs Scrum

- **Scrum:** columnas fijas, reiniciado cada Sprint.
- **Kanban:** columnas del proceso del equipo, flujo continuo sin fecha de cierre.

WIP Limits: por qué limitar el trabajo en progreso

Stop starting, start finishing. El principio central de Kanban.

Trabajo en progreso acumulado = inventario sin valor. No entrega nada al cliente hasta que se termina.

¿Qué es?

Un WIP limit de 3 en "Desarrollo" significa que no puede haber más de 3 tarjetas activas. Cuando se alcanza el límite, el equipo debe terminar algo antes de empezar.

La analogía del tráfico

Una autopista sin límite puede mover menos autos que una con límite. A cierta densidad el flujo colapsa. Igual con el WIP.

La lógica

Multitasking tiene costo cognitivo real. Alternar entre tareas reduce la productividad total. Terminar trabajo existente acelera el flujo global.

Sin WIP limits

Los desarrolladores trabajan en 4-5 cosas a la vez y no terminan ninguna. El cuello de botella se oculta: todo "está en progreso", nada está bloqueado.

WIP Limits: cómo definirlos

No hay fórmula universal. Se definen empíricamente y se ajustan con la experiencia.

Punto de partida

Regla de Anderson: WIP limit inicial = número de personas en el equipo para esa etapa. Un ítem por persona.

En la práctica

No se aplican a todas las columnas. Las más importantes son las de mayor capacidad. Pueden ser por persona o por columna.

Demasiado alto

La columna siempre está llena. Las tarjetas tardan en avanzar. El equipo hace multitasking frecuente.

Demasiado bajo

El equipo está bloqueado frecuentemente. Las personas están ociosas esperando que se libere capacidad.

El efecto sistémico

Cuando una columna llega al WIP limit, el equipo aguas arriba se detiene. Esto hace visible el cuello de botella y fuerza una conversación.

Referencia

Anderson: los WIP limits son "el corazón del método Kanban". Sin ellos, el tablero es solo una lista visual.

Cadencia continua vs. Sprints

Kanban no tiene iteraciones fijas. Se entrega cuando está listo, no cuando termina el sprint.

Criterio	Scrum	Kanban
Ritmo	Sprint fijo: 1-4 semanas	Flujo continuo, sin fecha de cierre
Planificación	Sprint Planning al inicio	Continua: se prioriza en cualquier momento
Entrega	Al final del Sprint	Cuando cada ítem está listo
Cambios	En el próximo Sprint	En cualquier momento respetando el WIP
Métricas	Velocidad (story points/sprint)	Lead time, cycle time, throughput
Mejor para	Desarrollo de producto iterativo	Soporte, operaciones, flujo impredecible

Métricas de flujo en Kanban

Lead time, cycle time y throughput. Miden el flujo, no el esfuerzo.

Tiempo de entrega Lead Time	Tiempo de ciclo Cycle Time	Rendimiento Throughput
<i>Tiempo total desde que el cliente solicita hasta que recibe.</i>	<i>Tiempo desde que el equipo empieza a trabajar hasta que entrega.</i>	<i>Cantidad de ítems completados por unidad de tiempo.</i>
Ejemplo • Día 1: cliente solicita 'exportar turnos a PDF'. • Día 8: feature en producción. • Lead time = 8 días.	Ejemplo • Equipo toma la tarea el día 5. • La completa el día 8. • Cycle time = 3 días.	Ejemplo • 8 features completadas en 2 semanas. Throughput = 4 features/semana.

Lo que percibe el cliente. Incluye tiempo en cola.

Lead time $\geq$ Cycle time siempre.

20 ítems pendientes $\div$ 4/sem = ~5 semanas.

Kanban vs. Scrum: cuándo usar cada uno

No son opuestos. Muchos equipos usan Scrumban. La elección depende del tipo de trabajo.

Usá Scrum cuando...

El trabajo es de producto con objetivos claros por iteración. El cliente puede participar en reviews periódicas.

La señal clave para Scrum

El equipo puede comprometerse a un Sprint Goal y protegerlo de interrupciones durante 2 semanas.

Scrum no funciona si...

Los sprints se cancelan frecuentemente. El backlog se modifica constantemente durante el sprint.

Scrumban

Tablero Kanban con WIP limits + ceremonias de Scrum. Equipos maduros que tienen cadencia propia.

Usá Kanban cuando...

El trabajo es soporte, operaciones o mantenimiento. Los ítems llegan en cualquier momento y el tamaño varía.

La señal clave para Kanban

Las interrupciones son la norma. La prioridad cambia varias veces por semana. Los ítems no tienen tamaño uniforme.

Kanban no funciona si...

No hay WIP limits: el tablero es solo una lista. No hay métricas. No hay reuniones de mejora.

El criterio final

Elegir el proceso que genera menos fricción para el tipo de trabajo que tiene el equipo.

Caso de adopción: Spotify

Scrum para producto, Kanban para operaciones. Un sistema híbrido a escala.

2008-2010

Scrum puro. Sprints de 2 semanas, todos los equipos sincronizados.

2011

30 equipos. La sincronización de sprints genera dependencias imposibles de gestionar.

2012

Autonomía por squad. Cada squad elige su proceso según el tipo de trabajo.

2013 →

Henrik Kniberg documenta el modelo. Se hace viral como "Spotify model". Adoptado globalmente.

Lección 1

No existe un proceso único correcto. El tipo de trabajo determina el proceso.

Lección 2

Kanban y Scrum no son excluyentes. Los equipos maduros terminan usando elementos de ambos.

Lección 3

El tablero sin WIP limits es decoración. El valor está en los límites y las conversaciones que generan.

Lección 4

La métrica que importa no es la velocidad del equipo: es el tiempo que tarda el valor en llegar al usuario.

Kanban aplicado: sistema de turnos médicos

¿Cómo sería el tablero del equipo que desarrolla y mantiene el sistema?

Backlog WIP: 3	Análisis WIP: 1	Desarrollo WIP: 2	Review WIP: 2	QA WIP: 1	Hecho sin límite
Módulo reportes	Login OAuth	API teleconsulta	Reg. paciente	Notif. 24hs	Modelo BD
Exportar PDF		Vista médico v2			Login básico
Filtro médico					

Review con WIP:2 es el cuello de botella: más cards en Desarrollo que capacidad de review. Señal para agregar un revisor o reducir WIP de Desarrollo.

Resumen

Lean es la filosofía. Kanban es el sistema operativo. Los WIP limits son el mecanismo.

Lean	Eliminar lo que no agrega valor. Los 7 desperdicios son el mapa.
Teoría de las restricciones	El sistema es tan rápido como su cuello de botella.
Mapa del flujo de valor	Hacer visible el flujo completo para identificar dónde se acumula la espera.
Kaizen	Mejora continua e incremental. La Retrospectiva de Scrum es su implementación formal.
Kanban	Tablero + WIP limits + métricas de flujo. Hace visible el cuello de botella.

Actividad de cierre

Diseño e implementación de un tablero Kanban para el sistema de turnos.

01

Diseñar el tablero

En grupos de 3 a 4 personas: definir columnas (mín. 5, máx. 7), asignar WIP limits justificando cada uno, identificar el cuello de botella probable, definir swimlanes para bugs / features / urgentes.

02

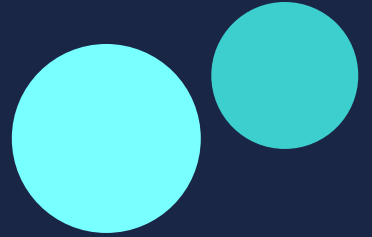
Identificar desperdicios

Sobre el tablero diseñado: identificar al menos 3 de los 7 desperdicios Lean que podrían ocurrir en ese flujo. Para cada uno: cómo se manifestaría y qué señal visual lo haría visible.

03

Presentación

Dos grupos presentan su tablero. El resto puede desafiar las decisiones de WIP limit.



Clase 14

Extreme Programming (XP)